{ "status": "analysis", "target": "api" }

CONFIDENTIAL

API Security Risk Analysis Report

JSONPlaceholder REST API Security Assessment

Target: jsonplaceholder.typicode.com

Assessment Date: March 23, 2026

Prepared By: Purnendu Rai

Scope: Read-Only Security Assessment

Table of Contents

1. Executive Summary

2. Assessment Overview

2.1 Scope and Objectives

2.2 Target API Information

2.3 Methodology

3. OWASP API Security Top 10

4. Risk Summary

5. Detailed Findings

5.1 Critical Severity

5.2 High Severity

5.3 Medium Severity

5.4 Low Severity

6. API Endpoint Analysis

7. Recommendations

8. Conclusion

1. Executive Summary

Key Findings at a Glance

This API Security Risk Analysis was conducted on the **JSONPlaceholder REST API**, a popular fake REST API used for testing and prototyping. The assessment identified **8 security vulnerabilities** across various categories, including one **CRITICAL** severity issue that would be catastrophic in a production environment.

8	1	3	6/10
Findings	Critical	High	OWASP Risks

Severity	Count	Risk Level
Critical	1	Immediate action required
High	3	Address within 30 days
Medium	3	Address within 90 days
Low	1	Address as resources permit

Most Critical Finding

API-006: Write Operations Allowed Without Authentication (CVSS: 9.1) - The API allows POST, PUT, and DELETE operations without any authentication, enabling complete data manipulation by anonymous users. In a production API, this would lead to catastrophic data integrity compromise.

OWASP API Security Top 10 Coverage

The assessment detected **6 out of 10** OWASP API Security risks, including:

- Broken Object Level Authorization (API1:2023)
- Broken Authentication (API2:2023)
- Broken Object Property Level Authorization (API3:2023)
- Unrestricted Resource Consumption (API4:2023)
- Broken Function Level Authorization (API5:2023)
- Security Misconfiguration (API8:2023)

2. Assessment Overview

2.1 Scope and Objectives

This API security assessment was conducted with the following scope and objectives:

Assessment Type	Read-Only API Security Assessment
Target API	JSONPlaceholder REST API
Base URL	https://jsonplaceholder.typicode.com
Assessment Date	March 22, 2026
Assessor	Future Interns - Cybersecurity Task 3
Methodology	OWASP API Security Top 10, Automated Testing, Manual Analysis

Scope Limitations

- Read-only requests (GET, safe POST where allowed)
- Public demo API endpoints only
- No exploitation or bypass attempts
- No flooding or DoS testing
- Documentation-based analysis

2.2 Target API Information

API Name	JSONPlaceholder
Description	Free fake REST API for testing and prototyping
Base URL	https://jsonplaceholder.typicode.com
Protocol	HTTPS
Data Format	JSON
Hosting	Cloudflare

Available Endpoints

Method	Endpoint	Description
GET	/posts	List all posts
GET	/posts/1	Get single post
GET	/posts/1/comments	Get post comments
POST	/posts	Create new post
GET	/users	List all users
GET	/users/1	Get single user
GET	/comments	List all comments
GET	/todos	List all todos
GET	/photos	List all photos

2.3 Methodology

The assessment followed industry-standard methodologies including the OWASP API Security Top 10 and security best practices:

Testing Approach

1. **Endpoint Discovery** - Identified all available API endpoints from documentation
2. **Authentication Testing** - Tested endpoints without authentication tokens
3. **Authorization Testing** - Attempted to access resources without proper authorization
4. **Data Exposure Analysis** - Analyzed response data for sensitive information
5. **Header Analysis** - Reviewed security headers in API responses
6. **Rate Limiting Assessment** - Tested for rate limiting controls
7. **Input Validation Testing** - Submitted various payloads to test validation

3. OWASP API Security Top 10

The OWASP API Security Top 10 is a standard awareness document for developers and API security professionals. It represents a broad consensus about the most critical security risks to APIs.

ID	Risk	Detected	Evidence
API1:2023	Broken Object Level Authorization	✓ **YES**	All user data accessible without authentication
API2:2023	Broken Authentication	✓ **YES**	No authentication required for any endpoints
API3:2023	Broken Object Property Level Authorization	✓ **YES**	Complete user objects with PII exposed
API4:2023	Unrestricted Resource Consumption	✓ **YES**	No pagination on large data endpoints
API5:2023	Broken Function Level Authorization	✓ **YES**	Write operations allowed without authorization
API6:2023	Unrestricted Access to Business Flows	✗ NO	Demo API - no business flows
API7:2023	Server Side Request Forgery	✗ NO	No URL parameters fetching remote resources

API8:2023	Security Misconfiguration	✓ **YES**	Missing security headers
API9:2023	Improper Inventory Management	✗ NO	Endpoints properly documented
API10:2023	Unsafe Consumption of APIs	✗ NO	No third-party API consumption observed

4. Risk Summary

The following table summarizes all identified security findings:

ID	Vulnerability	Severity	CVSS	OWASP
API-006	Write Operations Without Authentication	**CRITICAL**	9.1	API5:2023
API-005	No Authorization Checks for Resource Access	**HIGH**	8.2	API1:2023
API-001	No Authentication Required	**HIGH**	7.5	API2:2023
API-002	Excessive Data Exposure	**HIGH**	7.1	API3:2023
API-008	No Input Validation	**MEDIUM**	6.5	API3:2023
API-003	Missing Rate Limiting	**MEDIUM**	5.3	API4:2023
API-004	Missing Security Headers	**MEDIUM**	5.0	API8:2023
API-007	Server Information Disclosure	**LOW**	3.7	API8:2023

5. Detailed Findings

5.1 Critical Severity

API-006 Write Operations Allowed Without Authentication

CRITICAL

CVSS Score: 9.1 **OWASP:** API5:2023 - Broken Function Level Authorization

Description

The API allows POST, PUT, and DELETE operations without any form of authentication, enabling complete data manipulation by anonymous users. This represents a complete breakdown of access control for write operations.

Evidence

```
POST /posts → 201 Created (no auth required)
PUT /posts/1 → 200 OK (no auth required)
DELETE /posts/1 → 200 OK (no auth required)
```

Impact

Complete data integrity compromise. Attackers can create fake data, modify existing records, or delete all data without any restrictions. In a production API, this would be catastrophic and could lead to complete system compromise.

Remediation

1. Implement authentication for all write operations (POST, PUT, DELETE, PATCH)
2. Use API keys, OAuth 2.0, or JWT tokens for authentication
3. Implement role-based access control (RBAC) to restrict create/update/delete permissions
4. Validate user permissions before executing write operations
5. Log all write operations with user identification for audit purposes

5.2 High Severity

API-005 No Authorization Checks for Resource Access

HIGH

CVSS Score: 8.2 **OWASP:** API1:2023 - Broken Object Level Authorization

Description

Any client can access any user's data by simply changing the ID parameter in the URL. There is no verification that the requesting user owns or has permission to access the requested resource.

Evidence

```
GET /users/1 → Returns complete user data for user 1
GET /users/2 → Returns complete user data for user 2
GET /users/3 → Returns complete user data for user 3
(No authorization check performed)
```

Impact

Attackers can enumerate all user IDs and harvest complete user profiles. This is a horizontal privilege escalation vulnerability that allows access to other users' data. Combined with excessive data exposure, this reveals significant PII.

Remediation

1. Implement object-level authorization checks for every resource access
2. Verify the authenticated user has permission to access the requested resource
3. Use user-specific queries (e.g., WHERE user_id = current_user_id)
4. Implement resource ownership validation
5. Return 403 Forbidden for unauthorized access attempts

API-001 No Authentication Required for Any Endpoints

HIGH

CVSS Score: 7.5 **OWASP:** API2:2023 - Broken Authentication

Description

The API allows complete access to all resources without any form of authentication. This includes read access to sensitive user data and write operations that modify resources.

Evidence

```
All tested endpoints returned successful responses without:
- API keys
- Bearer tokens
- Session cookies
- Any authentication mechanism
```

Impact

Attackers can read, create, modify, and delete any data without authorization. In a production API, this would lead to complete data compromise and potential regulatory violations.

Remediation

1. Implement authentication mechanisms (API keys, OAuth 2.0, JWT)
2. Require authentication for all endpoints accessing sensitive data
3. Implement token expiration and refresh mechanisms
4. Use HTTPS for all authenticated requests
5. Implement proper session management

API-002 Excessive Data Exposure in API Responses

HIGH

CVSS Score: 7.1 **OWASP:** API3:2023 - Broken Object Property Level Authorization

Description

API responses include complete user objects with sensitive Personally Identifiable Information (PII) including email addresses, phone numbers, physical addresses, and geolocation coordinates.

Evidence

```
GET /users returns for each user:
- email: "Sincere@april.biz"
- phone: "1-770-736-8031"
- address: {street, suite, city, zipcode, geo: {lat, lng}}
- company: {name, catchPhrase, bs}
Total response: 5,645 bytes for 10 users
```

Impact

Exposure of PII violates privacy regulations (GDPR, CCPA). The exposed data can be used for identity theft, phishing attacks, social engineering, or sold on the dark web.

Remediation

1. Implement response filtering based on user roles and permissions
2. Use Data Transfer Objects (DTOs) to expose only necessary fields
3. Apply field-level authorization checks
4. Never expose sensitive fields like passwords, SSNs, or full addresses
5. Implement data masking for partially visible fields

5.3 Medium Severity

API-008 No Input Validation on POST/PUT Data

MEDIUM

CVSS Score: 6.5 **OWASP:** API3:2023 - Broken Object Property Level Authorization

Description

The API accepts and stores arbitrary JSON data in POST/PUT requests without validation of required fields, data types, or content sanitization.

Impact

Data quality issues, potential injection attacks if data is later processed without sanitization, storage of malicious payloads, and inconsistent data structure.

Remediation

1. Implement strict input validation using JSON Schema
2. Validate required fields, data types, and string lengths
3. Sanitize all input data before processing
4. Reject requests with unexpected fields
5. Implement content-type validation

API-003 Missing Rate Limiting on Data-Intensive Endpoints

MEDIUM

CVSS Score: 5.3 **OWASP:** API4:2023 - Unrestricted Resource Consumption

Description

While rate limiting headers are present (X-RateLimit-Limit: 1000), large data endpoints can be requested without pagination or throttling, allowing potential resource exhaustion.

Evidence

```
/comments endpoint returns 500 records (157,745 bytes) in a single request
/posts endpoint returns 100 records (27,520 bytes)
/todos endpoint returns 200 records (24,311 bytes)
No query parameter limits observed
```

Impact

API can be overwhelmed by large data requests leading to DoS conditions. Excessive bandwidth consumption increases costs and affects performance for legitimate users.

Remediation

1. Implement pagination with default limits (20-50 records per page)
2. Add query parameters for page size control with maximum limits
3. Enforce stricter rate limits on data-heavy endpoints
4. Implement request size limits
5. Use cursor-based pagination for large datasets

API-004 Missing Security Headers

MEDIUM

CVSS Score: 5.0 **OWASP:** API8:2023 - Security Misconfiguration

Description

API responses are missing critical security headers that protect against common web attacks and ensure secure communication.

Evidence

```
Missing headers in all responses:
✗ Strict-Transport-Security (HSTS)
✗ Content-Security-Policy
✗ X-Frame-Options
✗ X-XSS-Protection
✗ Proper CORS configuration
✓ X-Content-Type-Options: nosniff (present)
```

Impact

Increased vulnerability to XSS attacks, clickjacking, and man-in-the-middle attacks. Lack of HSTS allows SSL stripping attacks.

Remediation

1. Add Strict-Transport-Security: max-age=31536000; includeSubDomains
2. Add X-Frame-Options: DENY
3. Add X-XSS-Protection: 1; mode=block
4. Configure proper CORS policies
5. Add Content-Security-Policy for API documentation pages

5.4 Low Severity

API-007 Server Information Disclosure

LOW

CVSS Score: 3.7 **OWASP:** API8:2023 - Security Misconfiguration

Description

The Server header reveals Cloudflare as the hosting platform, which could aid attackers in targeting known vulnerabilities or platform-specific attacks.

Evidence

```
Server: cloudflare (present in all responses)
```

Impact

Information disclosure aids reconnaissance. Attackers can research Cloudflare-specific vulnerabilities or misconfigurations.

Remediation

1. Remove or obfuscate the Server header
2. Configure web server to not disclose version or platform information
3. Use generic server identifiers

6. API Endpoint Analysis

The following endpoints were tested during the security assessment:

Method	Endpoint	Status	Response Time	Size	Auth Required
GET	/users	200 OK	0.606s	5,645 bytes	No
GET	/users/1	200 OK	0.576s	509 bytes	No
GET	/posts	200 OK	0.623s	27,520 bytes	No

GET	/comments	200 OK	0.602s	157,745 bytes	No
GET	/todos	200 OK	0.600s	24,311 bytes	No
POST	/posts	201 Created	0.668s	109 bytes	No
PUT	/posts/1	200 OK	0.788s	95 bytes	No
DELETE	/posts/1	200 OK	0.657s	2 bytes	No

7. Recommendations

Immediate Actions (Critical - Within 24 Hours)

1. **Implement Authentication:** Require API keys or tokens for all endpoints accessing sensitive data
2. **Restrict Write Operations:** Immediately require authentication for POST, PUT, DELETE operations
3. **Enable Authorization Checks:** Verify users can only access their own resources

Short-Term Actions (High Priority - Within 30 Days)

1. **Implement Data Filtering:** Use DTOs to limit exposed fields based on user roles
2. **Add Pagination:** Implement pagination with default limits on all list endpoints
3. **Add Security Headers:** Configure HSTS, CSP, X-Frame-Options, and other security headers
4. **Implement Input Validation:** Use JSON Schema validation for all request payloads

Medium-Term Actions (Medium Priority - Within 90 Days)

1. **Implement Rate Limiting:** Enforce stricter rate limits on data-intensive endpoints
2. **Add Request Logging:** Log all API requests with user identification for audit purposes
3. **Implement API Versioning:** Version your API to manage changes and deprecations
4. **Add Monitoring:** Implement API monitoring and alerting for suspicious activity

Long-Term Improvements

1. **Security Audit:** Conduct regular security assessments and penetration testing

2. **Developer Training:** Provide secure API development training for developers
3. **API Gateway:** Consider using an API gateway for centralized security controls
4. **Threat Modeling:** Implement threat modeling for new API features

8. Conclusion

This API Security Risk Analysis of the JSONPlaceholder API identified **8 security vulnerabilities** ranging from Critical to Low severity. While JSONPlaceholder is intentionally a demo API designed for testing, the security issues identified represent real vulnerabilities that would be catastrophic in a production environment.

Key Takeaways

- **Authentication is fundamental:** Never allow access to sensitive data or operations without authentication
- **Authorization matters:** Verify users can only access resources they own or have permission to access
- **Limit data exposure:** Only return the data that is necessary for each use case
- **Validate all input:** Never trust client-provided data - validate everything
- **Implement defense in depth:** Layer multiple security controls for comprehensive protection

OWASP API Security Compliance

The assessment detected **6 out of 10** OWASP API Security Top 10 risks. Organizations should use the OWASP API Security Top 10 as a baseline for API security assessments and ensure their APIs address each risk category.

Final Recommendations

Organizations developing or deploying APIs should:

1. Conduct regular API security assessments
2. Follow the OWASP API Security Top 10 guidelines
3. Implement secure API development practices
4. Use API security testing tools in CI/CD pipelines
5. Monitor APIs for suspicious activity

References

- OWASP API Security Top 10:2023 - https://owasp.org/API-Security/
- OWASP API Security Project - https://github.com/OWASP/API-Security
- API Security Checklist - https://github.com/shieldfy/API-Security-Checklist
- JSONPlaceholder API - https://jsonplaceholder.typicode.com/

Prepared by: Future Interns - Cybersecurity Task 3

Date: March 22, 2026

Classification: Security Assessment Document